DM Tools

Session, Quest, and Story Flow Plan

A maintainable campaign workflow built around preparation, live play, resolution, and durable campaign memory.

DECISION	RECOMMENDATION
Core direction	Evolve DM Tools into a focused campaign-orchestration addon. Keep campaign facts in the host and keep specialized rules or combat logic in optional addons.
Hierarchy	Story arc -> quest -> scenario -> embedded scene/beat. A session references this structure; it does not own it.
Delivery	Build the domain and migrations first, then session prep, then derived graphs and optional integration APIs.

Architecture review and phased implementation plan | 26 July 2026

1. Executive plan

Recommended destination

DM Tools should become the DM's orchestration layer, not a second campaign database. It should connect intent before play to facts after play, while the host remains the durable source of truth for characters, places, events, mysteries, maps, backups, and synchronization.

What is already strong

- The host foundation is sufficient. DM-only collections, atomic multi-collection transactions, reviewed imports, lifecycle cleanup, graph facade, localization, and role-aware synchronization are already present.
- Core campaign entities are useful facts. Characters, factions, locations, events, mysteries, maps, and historical records should remain independent of planning tools.
- Addon boundaries are sound. DM Tools can work alone; sheets and compendiums can integrate through versioned APIs without becoming hard dependencies.

What currently blocks a coherent campaign flow

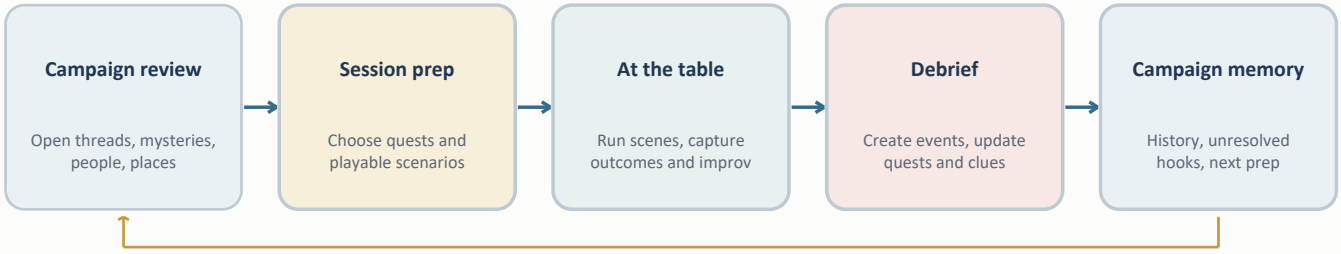
Gap	Why it matters	Design response
One flat scenario list	It cannot express long arcs, player goals, prerequisites, or session selection.	Add explicit arcs, quests, scenarios, and embedded beats.
Session is only an event number	Preparation, at-table notes, and post-session reconciliation have no shared workspace.	Add a DM-only session plan and resolution workflow.
Scenario graph has nodes but no edges	It is a catalog in graph form, not a decision aid.	Render only explicit semantic links and scoped subgraphs.
Planning and history are easy to mix	Future edits can rewrite what actually happened or turn events into speculative nodes.	Keep planned intent separate from canonical events.
No persisted schema migration path	The data model is still small, so this is the cheapest moment to establish safe evolution.	Version records and add additive, tested migrations before new UI.

Scope guardrails

- No database, framework, or infrastructure rewrite. JSON remains appropriate at this campaign scale.
- No generic everything-links-to-everything graph. Use domain fields and a small link vocabulary.
- No combat engine inside DM Tools. A future encounter addon may optionally consume the planning API.
- No automatic publication of secret DM notes. Player-facing quest information, if added, uses an explicit projection.

2. Design around the real D&D; loop

The primary unit of work is not a record type. It is the repeating cycle from campaign review to preparation, play, debrief, and renewed campaign memory.



The loop is deliberate: post-session facts feed the next preparation cycle.

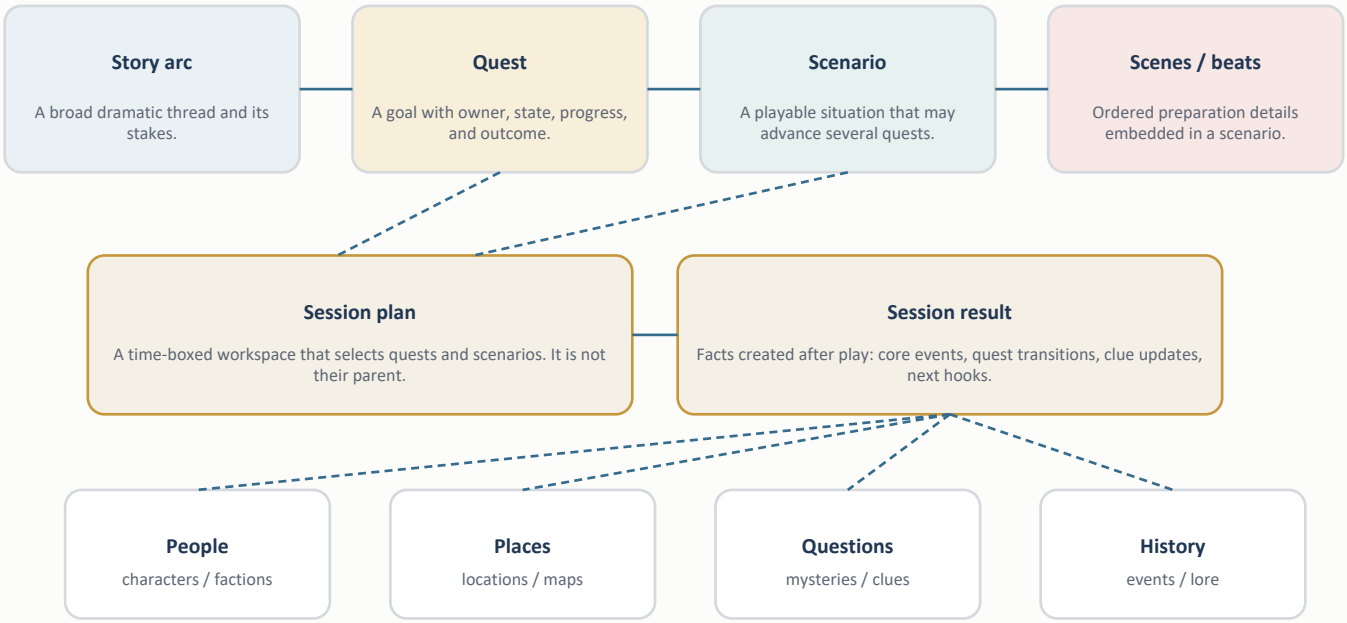
Stage	DM question	Tooling that should answer it	Durable result
Campaign review	What is unresolved and what matters next?	Active quests, mysteries, recent events, relationship and location context.	A shortlist for preparation.
Session prep	What situations might the players reach?	Session plan containing selected quests, scenarios, beats, participants, locations, and contingencies.	A versioned DM-only plan.
At the table	What do I need now, with minimal friction?	Compact run view, quick status changes, notes, links, and event drafts. No required graph interaction.	Temporary notes and explicit decisions.
Debrief	What actually happened and what changed?	Guided reconciliation: create core events, advance quests, reveal clues, set next hooks.	One atomic campaign update.
Between sessions	What new choices follow from the outcome?	Unresolved-thread dashboard and scoped story graph.	Next-session candidates.

Critical UX rule

The live-session surface must be faster than paper notes. Graphs are for review and preparation; the at-table view should be a focused agenda with one-click transitions and keyboard-friendly access to linked facts.

3. Proposed domain model

Use explicit concepts with different lifecycles. Avoid arbitrary recursive nesting and avoid treating the graph as stored truth.

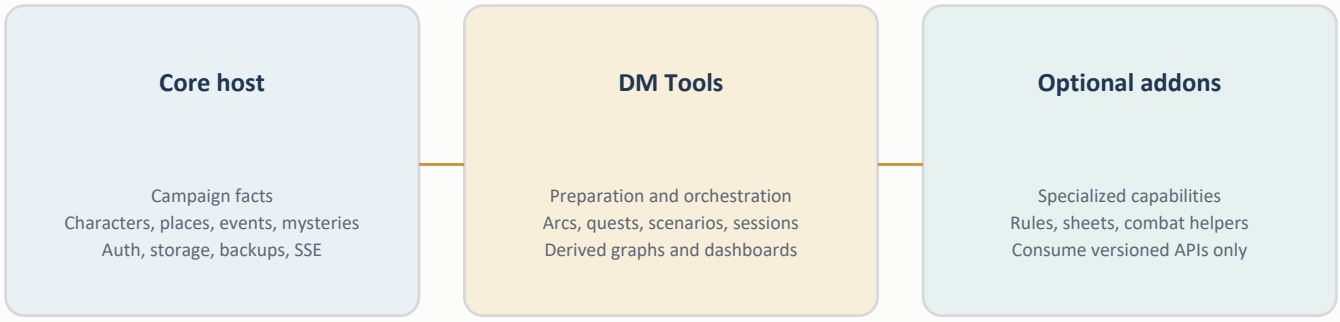


Concept	Owns	References	Lifecycle rule
Story arc	Premise, stakes, state, outcome summary	Quests, people, factions, places	Long-lived; closed when its dramatic question is resolved.
Quest	Goal, owner, state, progress, reward/outcome, public summary candidate	Arc, scenarios, mysteries, people, places	Player-facing objective semantics; may span many sessions.
Scenario	Situation, status, GM summary, risks, entry conditions, outcomes	Many quests and core entities	Reusable planning unit; not assumed to occur.
Scene / beat	Order, purpose, notes, optional completion state	Local references within its scenario	Embedded detail; not independently searchable until a real need appears.
Session plan	Date/number, agenda, selected scenarios, notes, state	Quests, scenarios, core entities	Cross-cuts the hierarchy; archived after reconciliation.
Core event	What happened, when, where, who was involved	Core entities and resulting planning records	Canonical fact after play; never a speculative scenario.

Minimal link vocabulary

- contains is represented by fields and embedded records, not a generic edge.
- leads-to is directed and optionally cycle-checked; it expresses a meaningful scenario transition.
- related is undirected and non-ordering; use it sparingly for alternate or parallel situations.
- Tags aid filtering but never create graph edges. Timestamps and list order also never imply narrative causality.

4. Tooling surfaces and ownership



Ownership is explicit. Integration is optional and contract-based.

Surface	Primary job	Show by default	Avoid
DM dashboard	Orient the DM in under 30 seconds.	Current session, active quests, unresolved mysteries, recent changes, blocked/ready scenarios.	All records, full graphs, speculative analytics.
Arc and quest board	Track campaign intent and progress.	State lanes, owner, next action, linked mysteries and scenarios.	Deep tree editors and arbitrary nesting.
Scenario editor	Prepare a playable situation.	Entry conditions, beats, participants, places, likely outcomes, explicit links.	Rules automation and hidden inferred dependencies.
Session prep	Assemble the next session without copying data.	Selected quests/scenarios, agenda, contingencies, reference links, checklist.	Duplicating character/location lore into notes.
Session run view	Support fast use during play.	Collapsed agenda, active beat, quick notes, event draft, reveal/update actions.	Mandatory forms, large visualizations, destructive shortcuts.
Debrief	Convert intent into campaign facts.	Outcome review, core event creation, quest transitions, clue changes, next hooks.	Silent automatic changes.
Story graph	Expose choices and dependencies.	One arc/quest/session scope, filters, list fallback, status/edge legend.	Whole-campaign graph as the default.
Import center	Bring in prepared structured plans safely.	Schema diagnostics, exact preview, atomic commit.	A second editing workflow or implicit overwrites.

Navigation recommendation

- Keep the stable /dm dashboard route as the entry point.
- Add /dm-quests, /dm-scenarios, and /dm-sessions; let each page deep-link to filtered graph views.
- Treat the graph as a view mode within a domain page, not a separate source of truth.
- Keep core entity navigation in the host; DM Tools links into it using existing routes and collection descriptors.

5. Storage, contracts, and safe evolution

The main architectural risk is not storage volume. It is allowing the planning model to grow without clear ownership, migration, and publication rules.

Collection	Shape	Access	Notes
storyArcs	Keyed	DM-only	Small, stable records; explicit state and outcome.
quests	Keyed	DM-only initially	Goal/progress semantics; references arcs and scenarios; prepare for explicit public projection.
scenarios	Keyed	DM-only	Migrate from the current list shape; embed ordered beats; preserve stable IDs.
scenarioLinks	List or keyed edges	DM-only	Source, target, type, label; validate ownership and dangling references.
sessionPlans	Keyed	DM-only	Cross-cutting workspace; state is draft, ready, running, reconciling, archived.

Record evolution

- Add schemaVersion to persisted planning records before introducing new shapes.
- Ship additive, deterministic migrations with fixtures for old, mixed, partial, and repeated-startup states.
- Keep import schema versioning separate from persisted schema versioning. Imports describe input; migrations protect stored campaign data.
- Use host transactions for multi-collection changes, especially debrief and migration publication.
- Centralize domain validation and status definitions so dashboard, graph, editor, server provider, and tests cannot drift.

Visibility and publication

> **Do not mix secrets and public quest text in one implicitly filtered record**
>
> Start with DM-only quests. If players later need a quest journal, add a separate public quest-card collection or explicit publish snapshot. Publication should copy only approved fields and should never expose DM notes, hidden links, prerequisites, or alternate outcomes.

Integration contract

- DM Tools remains independently useful and has no hard dependency on sheets or the compendium.
- Expose a small versioned read API only after the model stabilizes: active quest summaries, selected session scenarios, and linked core IDs.
- Optional consumers may enrich views but must degrade cleanly when absent. DM Tools never owns ruleset, combat, or character-sheet calculations.

6. Scalability, integrity, and maintenance

At expected campaign sizes, JSON and browser rendering are sufficient. Semantic and UI scale will fail before storage scale, so the plan emphasizes scoping, indexing, and diagnostics.

Concern	Recommendation	Reason
Graph growth	Default to arc, quest, or session scope; paginate/filter list fallback; never mount the whole campaign automatically.	The facade limit is generous, but a readable graph usually fails far below 1,000 nodes.
Dashboard growth	Show active/recent/blocked slices and counts. Link to full pages.	Rendering every scenario becomes slower and less useful as the campaign grows.
Change detection	Use collection revisions or a deterministic content digest, not counts plus newest timestamp.	Equivalent counts/timestamps can hide meaningful edits.
Reference integrity	Add read-only diagnostics for dangling IDs, duplicate IDs, invalid states, unreachable scenario links, and orphaned session references.	The current local dataset already contains a dangling location reference.
Location semantics	Separate semantic containment from map-placement context when local-map work resumes.	A single parent field cannot safely express both relationships.
Temporal fields	Separate user/source time from host audit revision where conflicts or ordering depend on it.	One updatedAt value should not simultaneously mean provenance, sort order, and concurrency.
Graph fallback	Keep the accessible list usable when Cytoscape is missing, unsupported, or fails.	Graph availability must not remove core planning functionality.

Operational invariants

- Every ID is stable across imports, migration, reinstall, backup, and restore.
- Every cross-record reference is validated on write and reported on read if legacy data is inconsistent.
- Every multi-record debrief is atomic: either all selected updates publish or none do.
- Every derived graph can be recreated from persisted records; graph layout is presentation state only.
- Every optional integration has a usable absent-provider state.
- Every state transition has an explicit user action and a localized failure path.

Deferred until demanded by real play

- Arbitrary recursive story trees, custom user-defined link types, automated quest progression, and predictive analytics.
- A full encounter/combat subsystem, initiative tracking, or rules adjudication inside DM Tools.
- Structured clue delivery beyond the existing mystery model, unless at-table use proves that free-form clues are insufficient.
- Server-side graph layout persistence and collaborative cursor state.

7. Phased implementation plan

The order is intentionally dependency-driven: define and protect the model before building screens that would freeze accidental semantics.

1

Domain foundation and migration

Build: Canonical status/validation module; schemaVersion; keyed scenarios; storyArcs, quests, scenarioLinks, sessionPlans; deterministic migrations; integrity diagnostics.
Gate: Old data survives restart and backup/restore; invalid references are reported; no UI behavior regresses.

2

Quest, arc, and scenario workflows

Build: CRUD pages; explicit references; embedded beats; state transitions; focused list/filter views; import schema update with exact preview.
Gate: A DM can model one real arc and prepare scenarios without using raw JSON or duplicate notes.

3

Session preparation and run view

Build: Session plan builder; agenda; selected quests/scenarios; quick notes; active beat; event drafts; keyboard and mobile behavior.
Gate: Preparing and running a session is faster than switching among separate pages or external notes.

4

Atomic debrief and campaign memory

Build: Guided reconciliation through F2 transactions; create core events; update quest/scenario states and mysteries; archive session; propose next hooks.
Gate: One confirmed action publishes a coherent result, and a failure leaves the campaign unchanged.

5

Scoped graph and dashboard refinement

Build: Explicit-edge graph; arc/quest/session scopes; status and edge legends; independent list fallback; revision-based refresh; active dashboard slices.
Gate: Graphs answer a specific planning question and remain useful with hundreds of records.

6

Optional integrations and player projection

Build: Versioned planning read API; optional encounter/sheet enrichments; explicit public quest-card publishing if desired.
Gate: Removing any optional addon leaves DM Tools functional; published player data contains no secret fields.

8. Decisions, tradeoffs, and acceptance criteria

Decision	Recommendation	Tradeoff accepted
Hierarchy depth	Use arc -> quest -> scenario -> embedded beat.	Less generic than a recursive tree, but far clearer to edit, migrate, query, and explain.
Session relationship	Session references quests/scenarios; it is not their parent.	The model needs explicit references, but reusable content and campaign continuity remain intact.
Planned vs actual	Keep scenarios/plans in DM Tools and outcomes in core events.	Debrief adds a deliberate step, preventing speculation from becoming history.
Graph storage	Persist semantic links only; derive nodes, labels, grouping, and layout.	Views may require recomputation, but data remains portable and deterministic.
Player quests	Start DM-only; publish explicit safe projections later.	The first release lacks a player quest journal, but avoids accidental secret leakage and coupled schemas.
Integration	Soft, versioned, capability-based APIs only.	Some enrichment is unavailable when providers are absent, but each addon remains maintainable and independently useful.

Release acceptance criteria

- Understandable: a new maintainer can explain the difference between an arc, quest, scenario, session, and event without reading implementation code.
- Recoverable: migration, transaction failure, addon disable/re-enable, restart, backup, and restore preserve campaign data and stable IDs.
- Fast in play: the live session view exposes the current agenda and quick capture without opening a graph or a multi-step editor.
- Traceable: a post-session event can be linked back to the quest/scenario it resolved, while the original plan remains available for review.
- Scoped: dashboards and graphs remain useful as data grows because they open on active, recent, or selected subsets.
- Optional: DM Tools works without sheets, compendium, or future combat tooling; integrations fail closed and degrade visibly.

Recommended immediate next batch

Implement Phase 1 only. Establish the canonical domain vocabulary, record schemas, migrations, centralized validation, and integrity diagnostics before changing the dashboard or graph. This creates the stable foundation for every later workflow and is the safest point to revise assumptions.

Review basis

This plan is based on a source-level audit of the host and DM Tools contracts, current planning/import/graph/dashboard implementations, reference documentation, and the local campaign data shape. It does not sample either remote production campaign. No campaign names, prose, record IDs, or personal data are included.